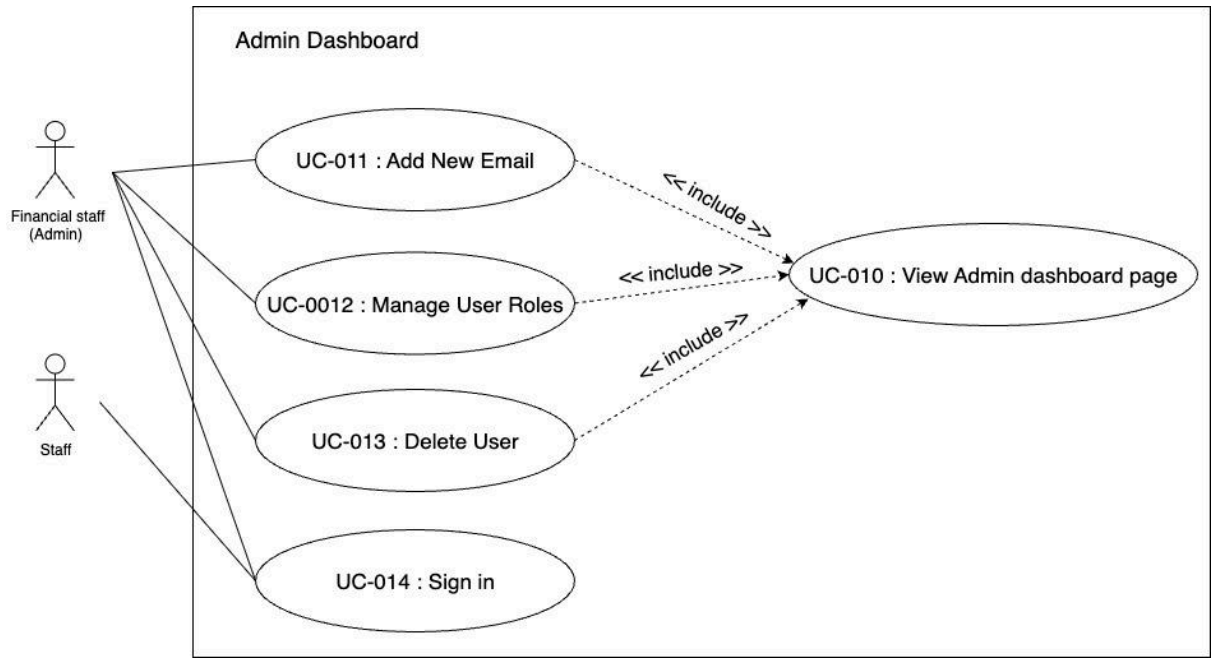
Use case: Admin Dashboard

This feature is a centralized control panel where financial staff can check and manage users. The ability is to add new CMU email addresses then the faculty staff can automatically log into the system and admin can manage the roles of users.

Usecase Diagram



Activity Diagram

Activity Diagram : Add Authorized Email

Activity Diagram : Manage User Roles.

Activity Diagram : Delete user

UseCase Description : View Admin dashboard page

Use case ID	UC-010		
Use case name	View Admin dashboard page		
Actors	Financial Staff (Admin)		
Description	Admin can view the user table to add users, change roles, or delete users, and can use filter functions to separate users staff or admin.		
Trigger	The process begins when the admin signs into the FAMIS system.The admin dashboard page is the main page for admin.		
Preconditions	Financial staff must be logged into the system and must be authenticated as an admin		
Use Case Input Specification			
Input	**Type**	**Constraint**	**Example**
CMU Email	Email	Must end with @cmu.ac.th	ABC@cmu.ac.th
Post Conditions	The user table shows the names of those who have already registered, sorted from admin to staff.		
Normal Flows			
User		**System**	
1. The admin opens the Admin Dashboard page.			
		2. The system retrieves a list of all registered users from the database, sorted by admin and then staff. If the number of lists of all registered users exceeds 20, the table includes pagination controls. [**E1**: Server error] [**E2**: Internal Server Error]	
		3. The system displays a “Add new user” button, search box, role filter function and a user table that contains all registered users. [**A1**: Have a search term in the search box] [**A2**: Filter function select a role]	
		4. A user table that contains all registered users, that indicates role, email, and department. Each row contains a "Change Role" icon and a "Delete" icon.	
Alternative Flow		[A1: Have a search term in the search box] • A1.1: The system will query a list of registered users.	

	[A2: Filter function select a role] ● A2.1: The system will display a list of registered users according to the selected filter, such as staff or admin only.
Exception Flow	[E1: Database Connection Error] ● E1.1: The system cannot establish a connection with the database. ● E1.2: The system will display an alert: "Error: Unable to retrieve document list from database. Please try again later" [E2: Internal Server Error] ● E2.1: The system encounters an unexpected error while querying for the document list. ● E2.2: The system will display an alert: "Error: An unexpected server error occurred while retrieving the document list. Please try again later."

URS-010: The admin views admin dashboard page.

SRS-054: The system retrieves a list of all registered users from the database, sorted by admin and then staff.
SRS-055: The “Admin dashboard” page displays a “Add new user” button, search box, role filter function and a user table that contains all registered users.
SRS-056: If the number of lists of all registered users exceeds 20, the table includes pagination controls.
SRS-057: The system must display a user table that contains all registered users, that indicate role, email, and department. Each row contains a "Change Role" icon and a "Delete" icon.
SRS-058: The system does not have a search term in the search box, the system displays all registered users.
SRS-059: The system has a search box with search terms, the system will query registered users.
SRS-060: The system does not use a role filter function, the system displays all registered users.
SRS-061: The system uses a role filter function, the system will display a list of registered users according to the selected filter, such as staff or admin only.

Usecase Description : Add Authorized Email

Use case ID	UC-011		
Use case name	Add New Email		
Actors	Financial Staff (Admin)		
Description	Admin can assign a new CMU email address and assign a role to that email, allowing access to the system.		
Trigger	The process begins when the admin clicks the “Add new user” button on the admin dashboard.		
Preconditions	Financial staff must be logged into the system and must be authenticated as an admin.		
Use Case Input Specification			
Input	**Type**	**Constraint**	**Example**
CMU Email	Email	Must end with @cmu.ac.th	ABC@cmu.ac.th
Role	Text	Admin or Staff	“Admin”
Post Conditions	The new CMU email and assigned role are stored in the database and activated for login.		
Financial Staff (Admin)		**System**	**CMU Email**
1. The admin opens the Admin Dashboard page.			
		2. The system retrieves a list of all registered users from the database, sorted by admin and then staff. [**E1**: Server error]	
		3.The system displays a user table that contains all registered users, including their role, email, and department. Each row contains a "Change Role" icon and a "Delete" icon. And display the “Add new user” button.	
4. The admin clicks “Add new user” button.			
		5. The system displays the Add new user page.	
6. The admin inputs the CMU email and assigns a role.			

7. The admin clicks “Confirm” button. [**A1**: Admin clicks “Cancel” button]		
	8. The system stores the CMU email and role in the database [**E2**: Database Failure]	
	9. The system displays an alert “[user@cmu.ac.th] is already registered.”	
		10. CMU Email must receive the email message “[user@cmu.ac.th] is already registered in the FAMIS system.” [**E3**: Email Send Failure]
Alternative Flow	[A1: Admin clicks “Cancel” button] ● A1.1 The system will cancel the add new user process ● A1.2 Returns to the dashboard without saving changes.	
Exception Flow	[E1: Server error] ● E1.1: The system displays an alert "Load data failed due to server error. Possible causes include connection timeout, internal server failure, or file overload. Please try again later." [E2: Database Failure] ● E2.1: The system displays an alert "Failed to store data. Please try again later." [E3: Email Send Failure] ● E3.1 The system displays an alert **"**CMU email add successfully, but failed to send notification email."	

URS-008: The admin can add new CMU email addresses to the system.

SRS-055: The “Admin dashboard” page displays a “Add new user” button, search box, filter function and a user table that contains all registered users.

SRS-062: Upon the admin clicks the “Add new user” button, the system navigates to the "Add new user" page.

SRS-063: The "Add new user" page contains an input box to enter the CMU email and text to select roles such as admin or staff with a “Confirm” button and a “Cancel” button.

SRS-064: The system verifies that the CMU email address entered ends with "@cmu.ac.th".

SRS-065: Upon clicking the "Confirm" button, the email and role are stored in the database.

SRS-066: The system stores the email and role success, the system will display an alert message " [user@cmu.ac.th] is email added successfully"

SRS-067: The system stores the email and role success, CMU Email must send mail to [user@cmu.ac.th] ,message
“ [user@cmu.ac.th] is already registered in the FAMIS system.”

SRS-068: Upon clicking the "Cancel" button returns to the dashboard without saving changes.

Use Case Description: Manage User Roles.

Use case ID	UC-012		
Use case name	Manage User Roles		
Actors	Financial Staff (Admin)		
Description	Admin can manage the system roles of users, change staff to admin or admin to staff.		
Trigger	The process begins when the admin accesses the Admin Dashboard page and initiates the role customization process.		
Preconditions	Financial staff must be logged into the system and must be authenticated as an admin.		
Use Case Input Specification			
Input	**Type**	**Constraint**	**Example**
CMU Email	Email	Must end with @cmu.ac.th	ABC@cmu.ac.th
Role	Text	Admin or Staff	“Admin”
Post Conditions	The updated user roles are stored in the database.		
Financial Staff (Admin)		**System**	
1. Financial staff(Admin) opens the Admin Dashboard page.			
		2. The system retrieves a list of all registered users from the database, sorted by admin and then staff. [**E1**: Server error]	
		3. The system displays a user table that contains all registered users, including their role, email, and department. Each row contains a "Change Role" icon and a "Delete" icon. And display the “Add new user” button.	
4. The admin clicks “Change role” icon.			
		5. The system will popup the change role window.On the popup, display the message [user@cmu.ac.th] and text to select roles such as admin or staff with a “Confirm” button and a “Cancel” button.	

6. The admin choose role and click “Confirm” button [**A1**: Financial staff(Admin) clicks “Cancel” button]		
	7. The system will update the role user in the database [**E2**: Database Failure]	
	8. The system displays an alert "[user@cmu.ac.th] role is successfully updated."	
		9. CMU Email must receive the email message “[user@cmu.ac.th] role is successfully updated in the FAMIS system.” [**E3**: Email Send Failure]
Alternative Flow	[A1: Financial staff(Admin) clicks “Cancel” button] ● A1.1 The system will cancel the popup change role window. ● A1.2 Returns to the dashboard without saving changes.	
Exception Flow	[E1: Server error] ● E1.1: The system displays an alert "Load data failed due to server error. Possible causes include connection timeout, internal server failure, or file overload. Please try again later." [E2: Database Failure] ● E2.1: The system displays an alert "Failed to store data. Please try again later." [E3: Email Send Failure] ● E3.1 The system displays an alert “Role updated successfully, but failed to send notification email."	

URS-009: The admin can manage user roles.

SRS-054: The system retrieves a list of all registered users from the database, sorted by admin and then staff.

SRS-055: The “Admin dashboard” page displays a “Add new user” button, search box, role filter function and a user table that contains all registered users.

SRS-057: The system must display a user table that contains all registered users, that indicate role, email, and department. Each row contains a "Change Role" icon and a "Delete" icon.

SRS-069: Upon the admin click the “Change role” icon, the system will retrieve [user@cmu.ac.th] and role to check that the correct user has been requested. Then the change role popup window.

SRS-070: The change role popup window, display the message [user@cmu.ac.th] and text to select roles such as admin or staff with a “Confirm” button and a “Cancel” button.

SRS-071: Upon clicking the "Confirm" button the user role is updated in the database.

SRS-072: The system updated the user role success, the system will display an alert message "[user@cmu.ac.th] role is successfully updated."

SRS-073: The system updated the user role success, CMU Email must send mail to [user@cmu.ac.th] , message “[user@cmu.ac.th] role is successfully updated in the FAMIS system.”

SRS-074: Upon clicking the "Cancel" button returns to the dashboard without saving changes.

Usecase Description : Delete User Account

Use case ID	UC-013		
Use case name	Delete User Account		
Actors	Financial Staff (Admin)		
Description	Admin can remove the users accounts in this platform.		
Trigger	The process begins when the admin accesses the Admin Dashboard page and initiates the delete user account process.		
Preconditions	Financial staff must be logged into the system and must be authenticated as an admin.		
Use Case Input Specification			
Input	**Type**	**Constraint**	**Example**
CMU Email	Email	Must end with @cmu.ac.th	ABC@cmu.ac.th
Post Conditions	The remove user account in the database		
Financial Staff (Admin)		**System**	**CMU Email**
1. The admin opens the Admin Dashboard page.			
		2. The system retrieves a list of all registered users from the database, sorted by admin and then staff. [**E1**: Server error]	
		3.The system displays a user table that contains all registered users, including their role, email, and department. Each row contains a "Change Role" icon and a "Delete" icon. And display the “Add new user” button.	
4. The admin clicks “Delete” icon.			
		5. The system will popup the delete user account window.On the popup, display the message “Confirm to delete [user@cmu.ac.th] account” with a “Confirm” button and a “Cancel” button.	
6. The admin click “Confirm” button [**A1**: The admin clicks “Cancel” button]			

	7. The system removes the user account in the database [**E2**: Database Failure]	
	8. The system displays an alert " [user@cmu.ac.th] account has been successfully removed"	
		9. CMU Email must receive the email message “[user@cmu.ac.th] has been successfully removed in the FAMIS system.” [**E3**: Email Send Failure]
Alternative Flow	[A1: Financial staff(Admin) clicks “Cancel” button] ● A1.1 The system will cancel the popup delete account window. ● A1.2 Returns to the dashboard without saving changes.	
Exception Flow	[E1: Server error] ● E1.1: The system displays an alert "Load data failed due to server error. Possible causes include connection timeout, internal server failure, or file overload. Please try again later." [E2: Database Failure] ● E2.1: The system displays an alert "Failed to store data. Please try again later." [E3: Email Send Failure] ● E3.1 The system displays an alert **"**User deleted successfully, but failed to send notification email.”	

URS-010: The admin can delete user accounts.

SRS-054: The system retrieves a list of all registered users from the database, sorted by admin and then staff.

SRS-055: The “Admin dashboard” page displays a “Add new user” button, search box, role filter function and a user table that contains all registered users.

SRS-057: The system must display a user table that contains all registered users, that indicate role, email, and department. Each row contains a "Change Role" icon and a "Delete" icon.

SRS-075: Upon the admin click the “Delete” icon, the system will retrieve [user@cmu.ac.th] to check that the correct user has been requested. Then the delete popup window.

SRS-076: The delete popup window displays the message “Confirm to delete [user@cmu.ac.th] account” with a “Confirm” button and a “Cancel” button.

SRS-077: Upon clicking the "Confirm" button the user account is already removed in the system.

SRS-078: The system removed the user account success, the system will display an alert message "[user@cmu.ac.th] account has been successfully removed"

SRS-079: The system removed the user account success, CMU Email must send mail to [user@cmu.ac.th] , message “[user@cmu.ac.th] has been successfully removed in the FAMIS system.”

SRS-080: Upon clicking the "Cancel" button returns to the dashboard without saving changes.

Usecase Description : Sign in

Use case ID	UC-014		
Use case name	Sign in		
Actors	Faculty Staff and Financial Staff (Admin)		
Description	Users must sign in using CMU email. CMU email addresses that can be used for sign in must be registered by an admin.		
Trigger	Clicking at “Sign in with CMU Account” button		
Preconditions	Open the FAMIS system.		
Use Case Input Specification			
Input	**Type**	**Constraint**	**Example**
CMU Email	Email	Must end with @cmu.ac.th	ABC@cmu.ac.th
Post Conditions	Registered email address is successfully signed in.		
Financial Staff (Admin)		**System**	**CMU OAuth**
1. The user opens the FAMIS system.			
		2. The login page displays the “Sign in with CMU Account” button. [**E1**: Server error]	
3. The user clicks the “Sign in with CMU Account” button.			
		4. The system generates PKCE values and redirects to CMU OAuth.	
			5. CMU OAuth displays the CMU login page.
6. The user completes authentication successfully. [**A1**: The user cancels login at CMU OAuth]			
			7. CMU OAuth redirects back to redirect_uri with authorization_code.
		8. The system exchanges the authorization_code + code_verifier at CMU OAuth /token to	

	obtainaccess_token (and id_token/refresh_token if provided). [**A2**: Authorization code invalid/expired or token exchange fails]	
	9. The system verifies tokens and reads user identity (email) from the database.	
	10.The system calls the backend to check the whitelist and role.	
	11. Backend validates that the email is registered and returns user account , department and role (admin or staff). [**A3**: CMU email is not registered]	
	12. The system stores tokens/user info in session	
	13. The system navigates admin to “Admin Dashboard” page or staff to “File Upload” page.	
	14. The sidebar system displays the signed in emails along with a logout button. [**A4**: The user logout from the system]	
Alternative Flow	[A1: The user cancels login at CMU OAuth] ● A2.1 System returns to the login page. [A2: Authorization code invalid/expired or token exchange fails] ● A3.1 System returns “Login failed. Please try again”. [A3: CMU email is not registered] ● A3.1 The system will display the message "[user@cmu.ac.th] is not registered" [A4: The user logout from the system] ● A4.1 The system will log that user out.	
Exception Flow	[E1: Database Connection Error] ● E1.1: The system cannot establish a connection with the database. ● E1.2: The system will display an alert: "Error: Unable to retrieve document list from database. Please try again later" [E2: Internal Server Error] ● E2.1: The system encounters an unexpected error while querying for the document list. ● E2.2: The system will display an alert: "Error: An unexpected server error occurred while retrieving the document list. Please try again later."	

URS-011: The user can sign in to FAMIS system

SRS-081: The “Sign in” page displays the “Sign in with CMU Account” button.

SRS-082: Upon clicking the “Sign in with CMU account” button, the system will use CMU OAuth to sign into the FAMIS system.

SRS-083: The system verifies the user’s email against the whitelist after token exchange. To ensure it has been successfully registered and navigates admin to “Admin Dashboard” page or staff to “File Upload” page.

- If the CMU email is not registered, an error pop-up will display the message "[user@cmu.ac.th] is not registered".

SRS-084: The sidebar system displays the signed in emails along with a logout button.

SRS-085: Upon clicking the “Log out” button, the system will log that user out.

Method declaration

Method ID: M1
Method Name: Load_User_List(page)

Description:

Retrieves a paginated list of all registered users, including each user's CMU email and current role. This method populates the Admin Dashboard table when the page is opened. Upon successful retrieval, it returns a status of 'success', an array of users that shows emails,roles and departments, the current page number, and the number of users per_page.

Input Parameters:

Parameter	Type	Description
auth_user_id	String	The user_id of the authenticated Admin caller.
page	Interger	Page number.
filter_role	String(optional staff,admin or none)	Users can filter by staff and admin.

Note: This value comes from the token and must correspond to a user whose role = “Admin.”

Return:

Field	Type	Description
status	String	"success"=The user list was retrieved successfully. "error"=No users array is returned. Instead, code and message explain the issue.
users	Array	Showing a list of all logged-in users which shows email, role and departments.
page	Interger	Page number.
per_page	Interger	Number of users per page.

- If the system fails to retrieve user data from the database will return:

{ "status": "error", "message": "Unable to retrieve user list from database. Please try again later." }

-If the provided filter_role is invalid (not 'Admin', 'Staff', or empty) will return:
{ "status": "error", "message": "Invalid filter role provided." }

Method ID: M2
Method Name: changeRole(userId, role)

Description:

Allows an authenticated Admin to change the role of a selected user. The new role can be assigned as either **"Admin"** or **"Staff"**. Upon a successful update, the system attempts to send a notification email to the affected user. The method returns the operation status ('success' or 'partial_success'), the user id of the affected user, the new_role, and a confirmation message.

Input Parameters:

Parameters	Type	Description
target_user_id	String	The user_id of the user whose role will be changed.
new_role	String	The new role to assign. Allowed values: "Admin" or "Staff".

Note: The authenticated Admin (auth_user_id) is verified from session/token internally. No need to include it in the request body.

Return:

Field	Type	Description
status	String	"**success**" = Role updated and email sent. "**partial_success**" = Role updated but email failed. "**error**" = Operation failed entirely.
user_id	String	The target_user_id of the affected user
new_role	String	The updated role after the change.
message	String	Text sending to the one who got upgraded or downgraded the roles.

- If the selected user cannot be found will return:
  { "status": "error", "message": "Selected user cannot be found. Please refresh the list and try again." }
- If the system fails to update the user role due to server/database error will return:
  { "status": "error", "message": "Unable to update user role due to server error. Please try again later." }
- If a non-Admin user attempts the action will return:
{ "status": "error", "message": "Unauthorized access." }
- If the provided new_role is not 'Admin' or 'Staff' will return:
{ "status": "error", "message": "Invalid role specified. Allowed values are 'Admin' or 'Staff'." }

Method ID: M3
Method Name: deleteUser(userId)

Description:

Allows an authenticated Admin to delete a user account from the system. This method permanently removes the user record from the database. Upon successful deletion, it returns a status of **'success'** and a message describing the result.

Input Parameters:

Parameter	Type	Description
target_user_id	String	The user_id of the user to be deleted.

Return:

Field	Type	Description
status	String	"success" = User deleted, "error" = Operation failed.
message	String	Message to describe the result “User account successfully deleted”.

- If the system fails to delete user data due to server/database error will return:
{ "status": "error", "message": "Unable to delete user due to server error. Please try again later." }

-If a non-Admin user attempts the action will return:
{ "status": "error", "message": "Unauthorized access." }

-If the selected target_user_id cannot be found will return:
{ "status": "error", "message": "Selected user cannot be found. Please refresh the list and try again." }

Method ID: M4
Method Name: assignUser(email, role, department)

Description:

Adds a new CMU email address and assigns a user role (**"Admin" or "Staff"**) for system access. This method stores the email and role in the database to enable future logins for that user. Upon success, it returns a status of 'success' and a message confirming that the email and role have been stored.

Input Parameters:

Parameter	Type	Description
cmu_email	String	The CMU email to add. Must end with `@cmu.ac.th`.
role	String	The user role to assign. Allowed values: “Admin”, “Staff”.

Note: The authenticated admin’s identity is verified from the session/token; no `auth_user_id` parameter is required.

Return:

Field	Type	Description
status	String	`"success"` = email+role stored successfully. "error" = Operation failed.
message	String	Showing message “The email and role has been stored in the database”.

- If the system fails to add user data due to server/database error will return:
{ "status": "error", "message": "Unable to store data due to server error. Please try again later."}

-If the provided cmu_email is already registered in the system will return:
{"status": "error", "message": "This email is already registered." }

- If the provided cmu_email format does not end with "@cmu.ac.th" will return:
{ "status": "error", "message": "Invalid email format. Must end with @cmu.ac.th." }

Unit testing

Test Data

Test data	Data
UTC-M1-TD-01	auth_user_id = user_01
UTC-M1-TD-02	auth_user_id = user_02
UTC-M1-TD-03	auth_user_id = admin_01
UTC-M1-TD-04	auth_user_id = admin_01,user_02
UTC-M1-TD-05	auth_user_id = []
UTC-M1-TD-06	auth_user_id = “user_01”, “user_02”, page= “1”

Unit Testing: M1 - Load_User_List()

Test ID	Scenario	Input	Expected result
UT-M1-01	**Successful Retrieval:** The user list is retrieved successfully.	auth_user_id: "user_01"	**Returns** { "status": "success", "users": [{chatchai_cha@cmu.ac.th}]}
UT-M1-02	**Unauthorized Access:** A non-Admin user attempts to retrieve the list.	auth_user_id: "user_02"	**Returns** { "status": "error", "message": "Unauthorized access."}.
UT-M1-03	**Database Error:** An error occurs while fetching from the database.	auth_user_id: “user_01”	**Returns** { "status": "error", "message": "Unable to retrieve user list from database. Please try again later." }
UT-M1-04	**Successful Retrieval with Filter:** filter with admin role.	auth_user_id: “user_01”, “user_02”	**Returns** { "status": "success", "users": [{chatchai_cha@cmu.ac.th}, {patara_nun@cmu.ac.th}]}.
UT-M1-05	**Successful Retrieval with Empty Result**	auth_user_id: “....”	**Returns** { "status": "success", "users": [{}]}
UT-M1-06	**Pagination Test:** Test the next page data retrieval	auth_user_id: “user_01”, “user_02” page: “1”	**Returns** {"status": "success", "page": [1]}

Test Data

Test data	Data
UTC-M2-TD-01	auth_user_id = admin_01, target_user_id = user_02, new_role = "Admin"
UTC-M2-TD-02	auth_user_id = admin_01, target_user_id = user_999
UTC-M2-TD-03	auth_user_id = admin_01, target_user_id = user_02
UTC-M2-TD-04	auth_user_id = admin_01, target_user_id = user_02, new_role = "Admin"
UTC-M2-TD-05	auth_user_id = staff_01
UTC-M2-TD-06	auth_user_id = admin_01, target_user_id = user_02, new_role = "Guest"

Unit Testing: M2 - changeRole(userId, role)

Test ID	Scenario	Input	Expected result
UT-M2-01	**Successful Role Change:** A user's role is successfully updated in the database.	target_user_id: "user_02", new_role: "Admin"	**Returns** { "status": "success", "User_id": "user_02", "new_role": "Admin", "message": "User role has been successfully updated." }
UT-M2-02	**User Not Found:** An attempt is made to change the role of a non-existent user.	target_user_id: "user_03", new_role: "Admin"	**Returns** { "status": "error", "message": "Selected user cannot be found. Please refresh the list and try again." }
UT-M2-03	**Database Error:** An error occurs while fetching from the database.	target_user_id: "user_02", new_role: "Admin"	**Returns** { "status": "error", "message": "Unable to update user role due to server error. Please try again later." }
UT-M2-04	**Partial Success:** The role is updated, but the email notification fails to send(Mock EmailService fail).	target_user_id: "user_02", new_role: "Admin"	**Returns** { "status": "partial_success", "User_id": "user_02", "new_role": "Admin", "message": "Role updated, but failed to send notification email." }
UT-M2-05	**Unauthorized Access:** A non-Admin user attempts to change a user's role.	auth_user_id: "user_staff_01” target_user_id: "user_01" new_role: "Admin"	**Returns** {"status": "error", "message": "Unauthorized access."}.
UT-M2-06	**Invalid Role Input:** An attempt is made to assign an invalid role.	target_user_id: "user_02" new_role: "Guest"	**Returns** {"status": "error", "message": "Invalid role specified."}.

Test Data

Test data	Data
UTC-M3-TD-01	auth_user_id = admin_01, target_user_id = user_03
UTC-M3-TD-02	auth_user_id = staff_01, target_user_id = user_05
UTC-M3-TD-03	auth_user_id = admin_01, target_user_id = user_04
UTC-M3-TD-04	auth_user_id = admin_01 ; target_user_id = user_999

Unit Testing: M3 - deleteUser(userId)

Test ID	Scenario	Input	Expected result
UT-M3-01	**Successful Deletion:** A user account is successfully deleted from the database.	target_user_id: "user_03"	**Returns** {"status": "success", "message": "User account successfully deleted"}
UT-M3-02	**Unauthorized Access:** A non-Admin user attempts to retrieve the list.	target_user_id: "user_05"	**Returns** { "status": "error", "message": "Unauthorized access." }
UT-M3-03	**Database Error:** An error occurs while fetching from the database.	target_user_id: "user_04"	**Returns** { "status": "error", "message": "Unable to delete user due to server error. Please try again later." }.
UT-M3-04	**User Not Found:** An attempt is made to delete a non-existent user.	target_user_id: "user_999"	**Returns** {"status": "error", "message": "Selected user cannot be found."}.

Test Data

Test data	Data
UTC-M4-TD-01	auth_user_id = admin_01, cmu_email = "new.user@cmu.ac.th", role = "staff"
UTC-M4-TD-02	auth_user_id = admin_01, cmu_email = "existing.user@cmu.ac.th", role = "staff"
UTC-M4-TD-03	auth_user_id = admin_01, cmu_email = "test@gmail.com", role = "staff"
UTC-M4-TD-04	auth_user_id = admin_01, cmu_email = "new.user2@cmu.ac.th", role = "staff"

Unit Testing: M4 - Add_user_email()

Test ID	Scenario	Input	Expected result
UT-M4-01	**Successful Assignment:** A new email and role are successfully stored in the database.	cmu_email: "new.user@cmu.ac.th", role: "staff"	**Returns** { "status": "success", "message": "The email and role has been stored in the database" }.
UT-M4-02	**Email Already Exists:** An attempt is made to add an email that is already registered.	cmu_email: "chatchai_cha@cmu.ac.th", role: "staff"	**Returns** { "status": "error", "message": "This email is already registered." }
UT-M4-03	**Invalid Email Format:** An attempt is made to add an email that does not end with "@cmu.ac.th".	cmu_email: "test@gmail.com", role: "staff"	**Returns** { "status": "error", "message": "Invalid email format. Must end with @cmu.ac.th." }.
UT-M4-04	**Database Error:** An error occurs while storing the new email and role.	cmu_email: "new.user2@cmu.ac.th", role: "staff"	**Returns** { "status": "error", "message": "Unable to store data due to server error. Please try again later."}

UAT Testing

UAT-001: Verify Admin can view the user management dashboard

: This test case verifies that an authenticated Admin can access the Admin Dashboard and view the list of registered users, which corresponds to use case **UC-001: View Admin dashboard page** and requirement.

Precondition:

- The tester must be logged into the system with an account that has "Financial Staff (Admin)" privileges.
- At least one user account must exist in the system to be displayed in the user table.

Test steps:

1. The tester logs into the system.
2. The tester clicks on the "Admin Dashboard" page.

Expected Result:

- The system must display the "Admin Dashboard" page.
- The page includes an "Add new user" button and a filter function.
- A user table is displayed containing columns for "Role", "Email", and "Department".
- Each row in the user table displays a "Change Role" icon and a "Delete" icon.
- The users in the table are sorted by role, with "Admin" users appearing before "Staff" users.

UAT-002: Verify Admin can successfully add a new user

: This test case validates that an Admin can add a new user to the system as described in **UC-002:** Add New Email and **URS-002**.

Precondition:

- The tester is logged in as an Admin and is on the Admin Dashboard page.
- The email to be added (new.tester@cmu.ac.th) does not already exist in the system.

Test steps:

1. The tester clicks the "**Add new**" button.
2. The system navigates to the "**Add new user**" page.
3. The tester enters the email “**new.tester@cmu.ac.th**” into the input field.
4. The tester selects the "**Staff**" role from the dropdown options.
5. The tester clicks the "**Confirm**" button.

Expected Result:

- The system displays a success alert message, such as"**[new.tester@cmu.ac.th] is email added successfully**".
- The system returns to the Admin Dashboard page.
- The new user **“new.tester@cmu.ac.th”** appears in the user table with the assigned "Staff" role.

UAT-003: Verify the system prevents adding a duplicate user email

: This test case ensures the system handles the exception flow of **UC-002** where an Admin attempts to add a user whose email is already registered.

Precondition:

- The tester is logged in as an Admin.
- A user with the email **“admin.user@cmu.ac.th”** already exists in the system.

Test steps:

1. The tester navigates to the "Add new user" page.
2. The tester enters the existing email “admin.user@cmu.ac.th” into the input field.
3. The tester selects any role.
4. The tester clicks the "Confirm" button.

Expected Result:

- The system must display an alert message stating **"[existing.user@cmu.ac.th]** is already registered.".
- The tester remains on the "Add new user" page, allowing for corrections.

UAT-004: Verify Admin can change a user's role

: This test case validates that an Admin can modify a user's role as described in **UC-003: Manage User Roles** and **URS-003**.

Precondition:

- The tester is logged in as an Admin and is on the Admin Dashboard page.
- A user with the email “**new.tester@cmu.ac.th”** exists with the current role of "Staff".

Test steps:

1. The system displays a success alert message, such as "**[new.tester@cmu.ac.th]** role is successfully updated."
2. The system displays a pop-up window for changing the role.
3. In the pop-up, the tester selects the new role "Admin".
4. The tester clicks the "Confirm" button.

Expected Result:

- The system displays an alert message such as **"[new.tester @cmu.ac.th]** role is successfully updated."
- The pop-up window closes, and the role for **new.tester@cmu.ac.th** in the user table is updated from "Staff" to "Admin"

UAT-005: Verify Admin can delete a user account

: This test case verifies that an Admin can remove a user account from the system, as described in **UC-004: Delete User** and **URS-004.**

Precondition:

- The tester is logged in as an Admin and is on the Admin Dashboard page.
- A user with the email new.tester@cmu.ac.th exists in the system.

Test steps:

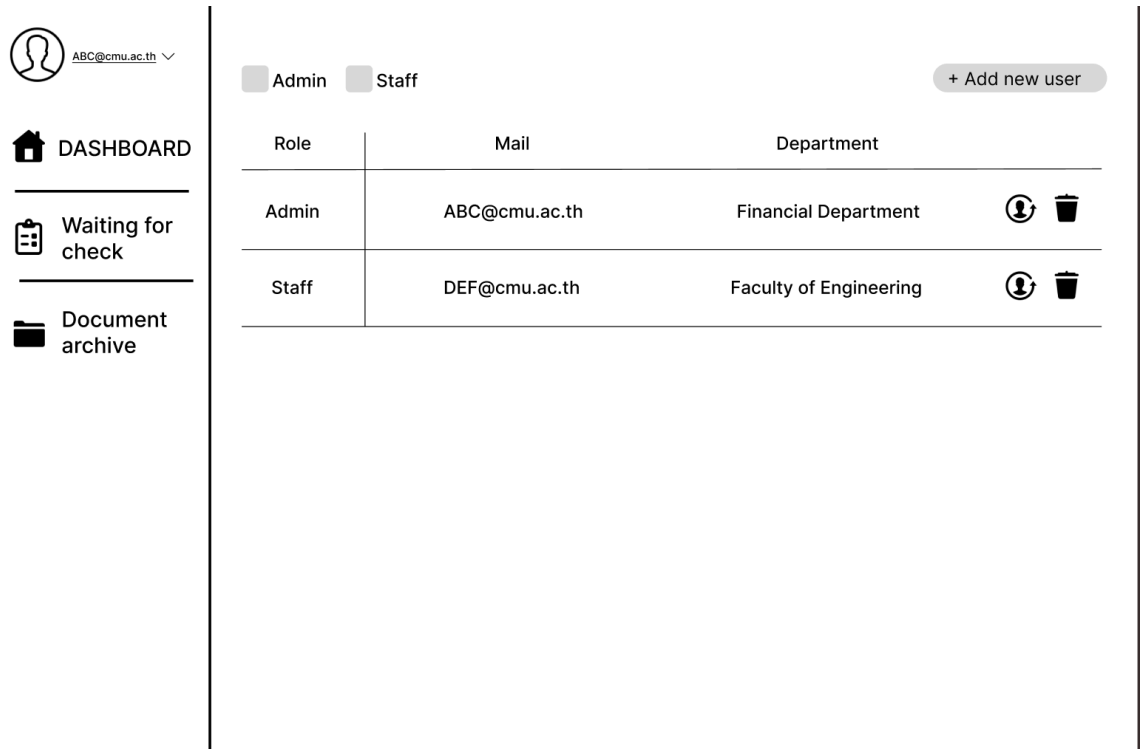
1. In the user table, the tester locates the user “**new.tester@cmu.ac.th** and clicks the "**Delete" icon.**”
2. The system displays a confirmation pop-up with a message like "Confirm to delete **[new.tester@cmu.ac.th]** account"
3. The tester clicks the "Confirm" button in the pop-up.

Expected Result:

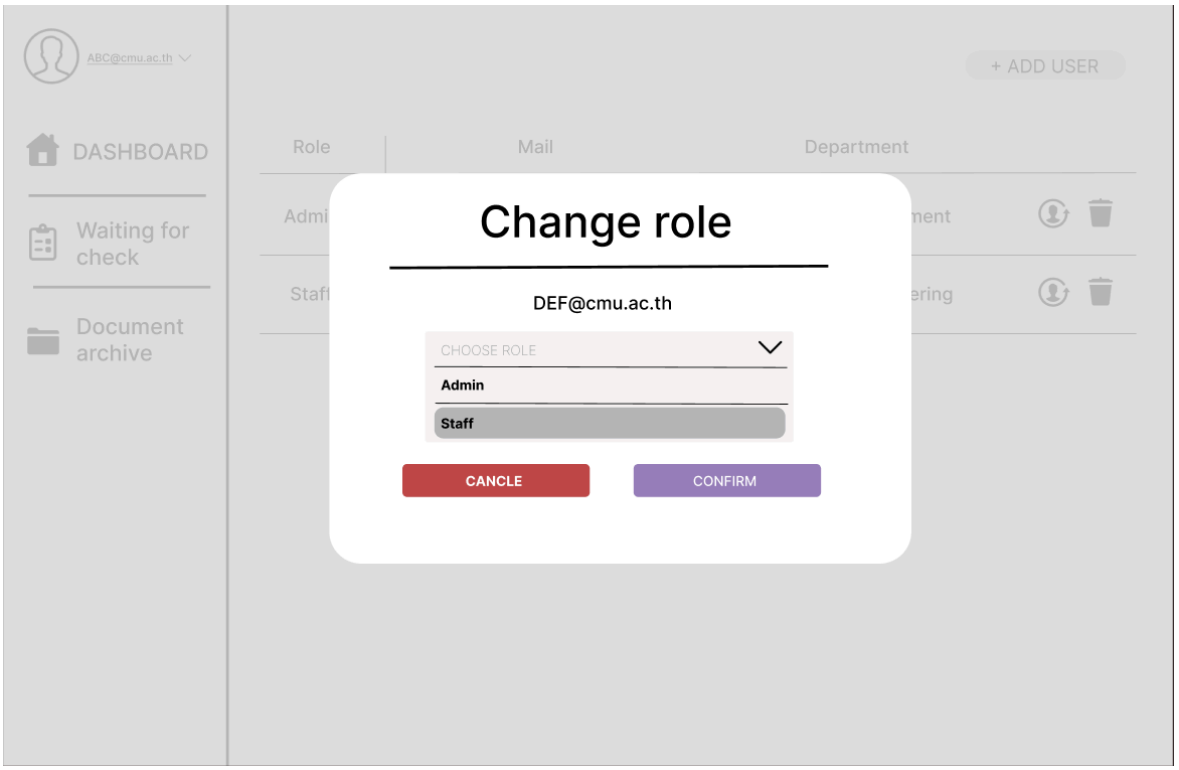
- The system displays an alert message such as "**[new.tester@cmu.ac.th]** account has been successfully removed".
- The user account for **[new.tester@cmu.ac.th]** is removed from the admin dashboard.

UI(Prototype)

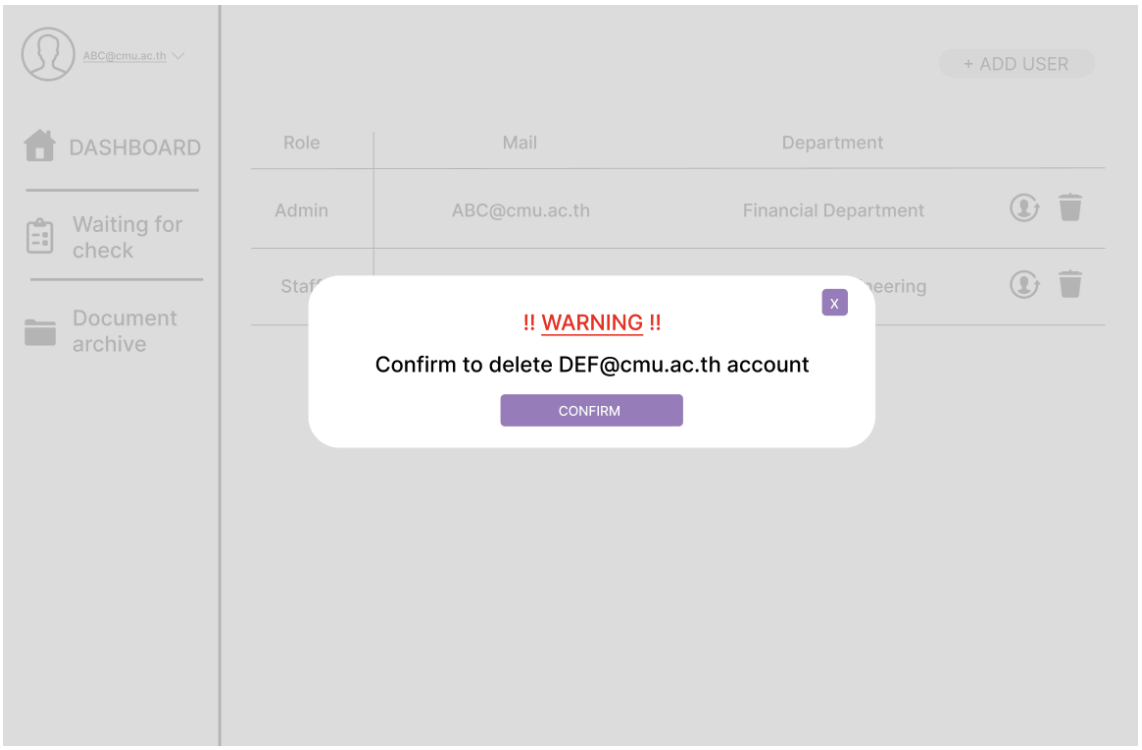
UI-001: Admin dashboard



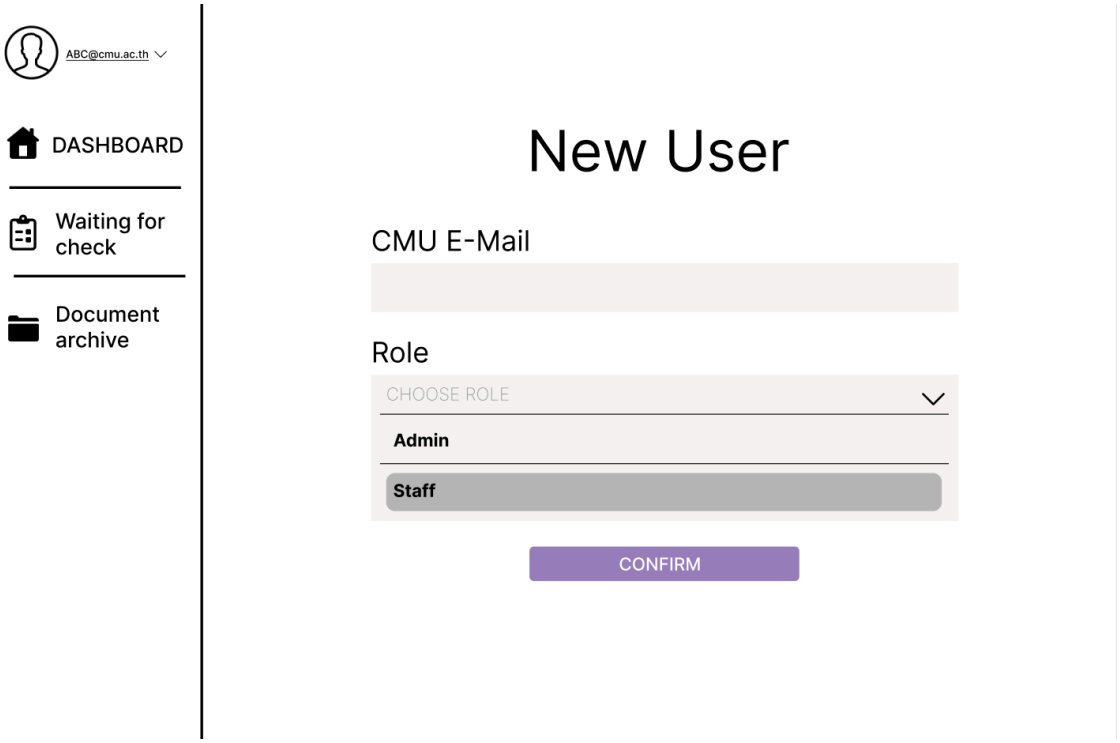
UI-002: Change role



UI-003: Delete user

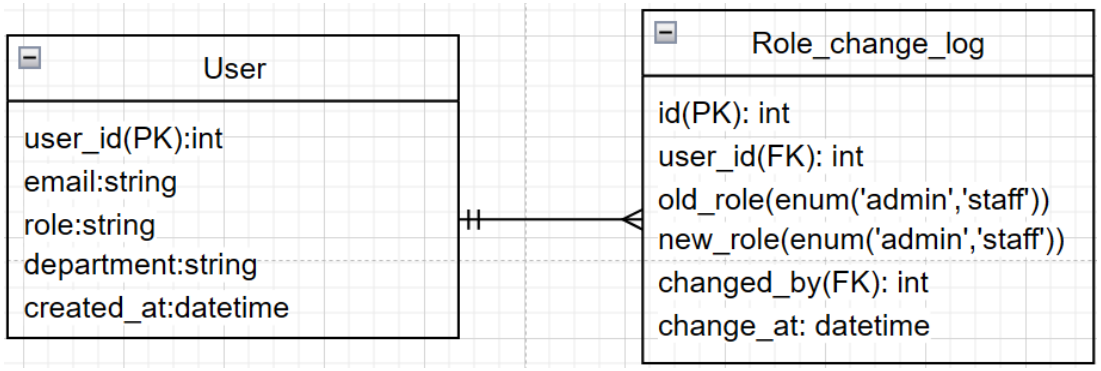


UI-004: Assign email(Admin)

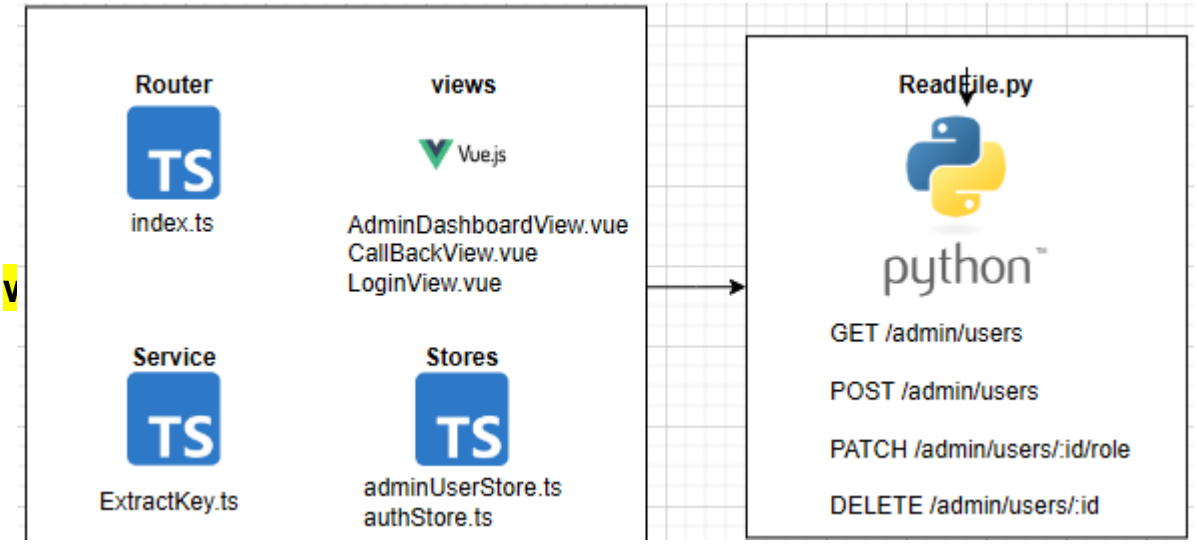


UI-005: Login page

ER-Diagram



Code Structure



Data Structure

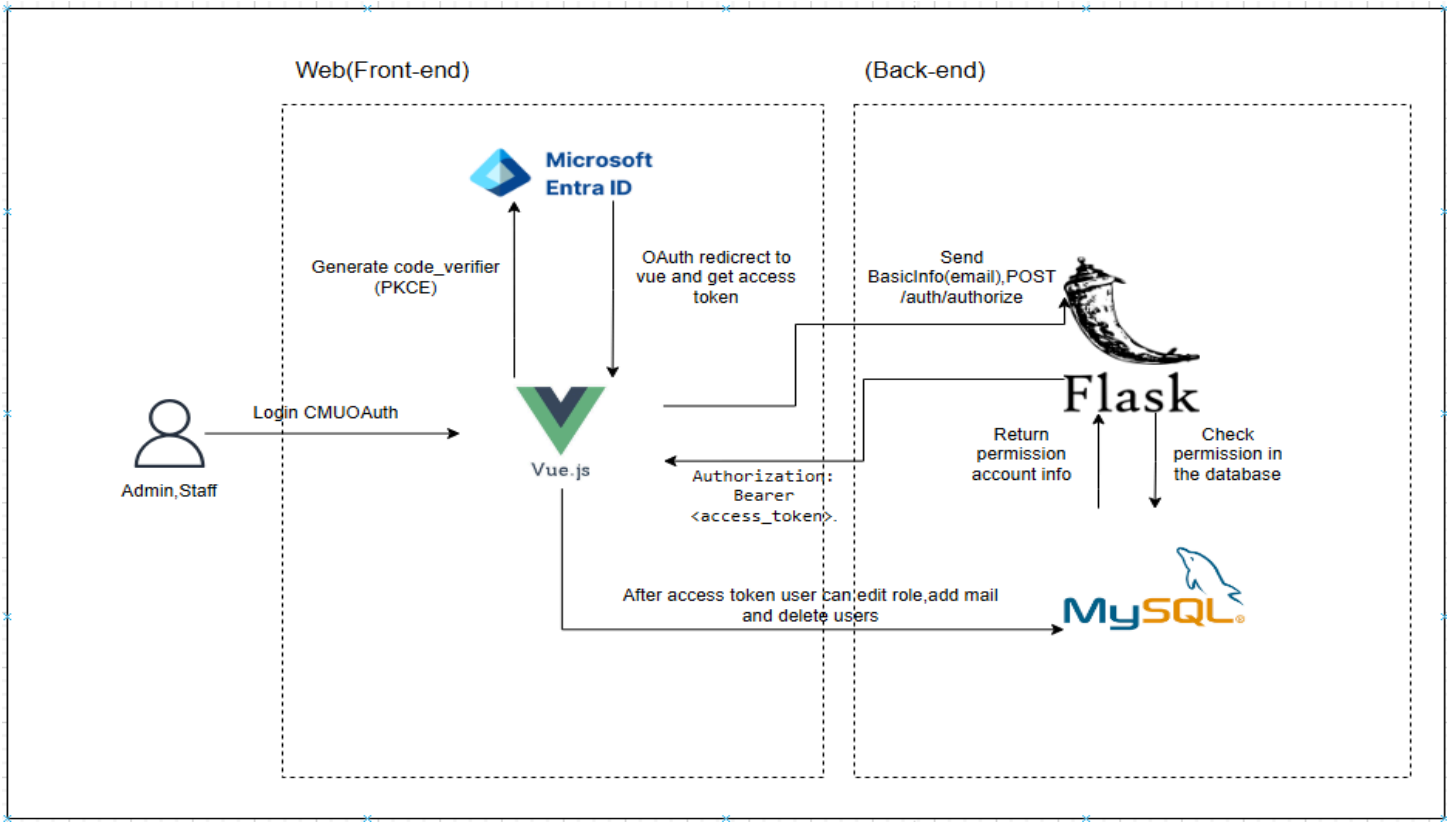
1. Table: User

Name	Type	Description	Example
user_id (PK)	int	A unique identifier for each user (Primary Key)	1
email	str	The user's email address, used for login and notifications.	chatchai.c@cmu.ac.th
department	str	The department or faculty the user belongs to	College of Arts, Media and Technology
role	str	The user's role in the system ('Faculty Staff', 'Financial Staff')	'Admin'
created_at	datetime	Date the assign	2025-06-23 10:30:00

2. Table: role_change_log

Name	Type	Description	Example
id (PK)	int	A unique identifier for each user (Primary Key)	1
user_id (FK)	int	The user's that has change role	patara_nun@cmu.ac.th
old_role	enum('admin', 'staff')	role	staff
change_by (FK → user.user_id)	int	Name of admin that change roles	2
created_at	datetime	Date the assign	2025-06-23 10:30:00

System Architecture



The process begins when an Admin or Staff user initiates a login request through the Vue.js web application. Vue.js generates a PKCE code_verifier and redirects the user to the Microsoft Entra ID authorization endpoint. After the user successfully authenticates, Microsoft Entra ID redirects back to the Vue.js application with an authorization code.

Vue.js then exchanges this authorization code along with the previously generated code_verifier for an access token (and optionally an id_token) from the Entra ID token endpoint. Using the access token, Vue.js may call Microsoft Graph to retrieve basic profile information such as email and department.

Once the required profile information is obtained, Vue.js sends it via a REST API call (**POST /auth/authorize**) to the Flask back-end. Flask verifies the user's account against the MySQL database, checks their role and permissions, and returns an authorization response containing the user's role and account information.

With the access token and role information stored in the session, the user can now access protected routes in the application. For admin users, Vue.js uses the access token to call Flask's admin APIs (e.g., add user, edit role, or delete user). Flask validates these requests against the database and responds with the results, which Vue.js displays back to the user.